Applied Intelligence (2023) 53:28848–28864
https://doi.org/10.1007/s10489-023-05043-z

NLINQ: A natural language interface for querying network performance

Barun Kumar Saha[1] · **Paul Gordon**[2] · **Tore Gillbrand**[2]

Accepted: 25 September 2023 / Published online: 18 October 2023

Abstract
Artificial Intelligence is finding increased applications in communication networks. In particular, the field of text-to-Structured Query Language (SQL) translation has great potential to improve customer experience by allowing the querying of network performance databases using natural language. Such adoption, however, is challenging, in general. On one hand, live production systems may have databases with non-semantic table and column names, which makes natural language parsing and text-to-SQL translation difficult. On the other hand, noisy input texts may lead to the generation of incorrect queries. Moreover, inaccurate transcription of speech input into text may further aggravate the problem. Motivated by these aspects, we investigate the problem of natural language-based querying of network performance databases used by Wireless Mesh Networks (WMNs). In particular, we fine-tune a state-of-the-art model to translate natural language questions into appropriate SQL queries. In order to mitigate the problem of non-semantic names, we generate database views with semantic column names, based on the existing tables. In addition, we make domain-specific corrections in the text in order to help generate accurate queries. We also design the Natural Language Interface for Network Query (NLINQ) prototype for a real-life industrial WMN solution. The results of the performance evaluation indicate that natural language text can be translated into SQL queries with an accuracy of 89.021–92.663%, on average. Moreover, the average turnaround time of NLINQ ranges between 1.263–2.013 seconds. The results indicate that NLINQ is suitable for real-time, interactive querying of network performance databases.

Keywords Database · Structured query language · Natural language processing · Network management · Wireless mesh networks · Text-to-SQL translation

1 Introduction

The recent advances in Artificial Intelligence (AI) and Natural Language Processing (NLP) [1–7] allow users to interact with systems using natural language text and speech. Such natural language interactions can be leveraged to fulfill different objectives, for example, querying the system and modifying its state or parameters. Consequently, Natural Language Interfaces (NLIs) [2] can complement the traditional graphical user interfaces (GUIs) and command-line interfaces (CLIs), as well as improve customer experience, for example, by reducing cognitive load. Today, the use of NLIs can be found across diverse domains, such as network management [2], data visualization [4], and mobile interactions [6].

In Intent-based Networks (IBNs) [8, 9], NLIs are typically used to specify network management instruction using natural language. However, IBNs largely remain unexplored in the context of wireless networks. In particular, Wireless Mesh Networks (WMNs) [10–12]—which provide wireless connectivity over large regions, together with fault-tolerance and self-organization capabilities—continue to evolve as a key technology in the industrial context. This is especially true in the utilities sector, where last-mile connectivity in power grids is often achieved using WMNs [13]. Although WMNs

✉ Barun Kumar Saha
barun.kumarsaha@hitachienergy.com

Paul Gordon
paul.gordon@hitachienergy.com

Tore Gillbrand
Tore.Gillbrand@hitachienergy.com

[1] Grid Automation R&D, Hitachi Energy, Bangalore 560048, India

[2] Grid Automation R&D, Hitachi Energy USA Inc., 3055 Orchard Drive, San Jose, CA 95134, USA

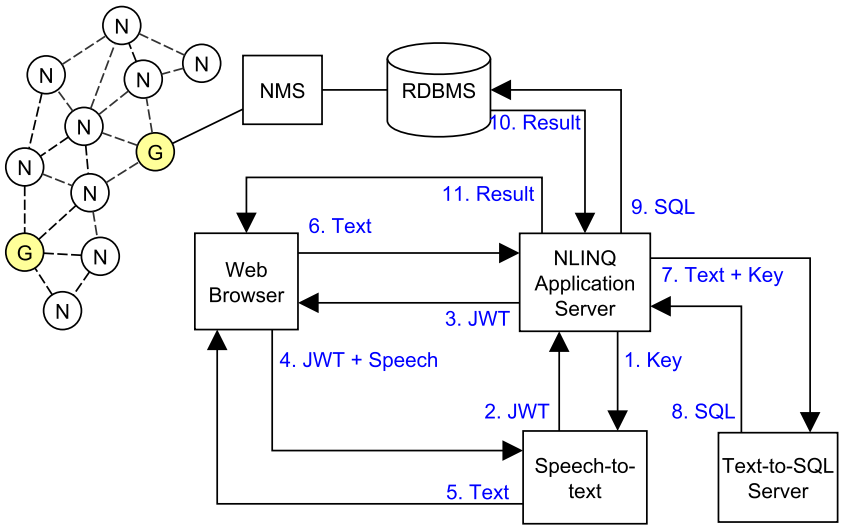

Fig. 1 The architecture of NLINQ, depicts communication flows among its different components. The circles labeled with "N" indicate mesh nodes; those labeled with "G" indicate mesh gateways. The dashed lines indicate wireless links. Users query the network performance using a Web browser together with speech inputs

and many modern networks largely operate autonomously, occasional under-performance, for example, due to interference, cannot be ruled out. Therefore, given the scale and distributed nature of WMNs (or networks, in general), an NLI-based approach to query the performance measurement data, such as throughput, latency, and signal-to-noise ratio (SNR), and precisely fetch the intended information can potentially help in quicker fault detection, troubleshooting, and resolution. Typically, periodic performance measurement data for such networks are stored in a relational database, which can be queried using Structured Query Language (SQL).

The contemporary use of NLIs in the context of network management, however, is largely focused on issuing commands to take actions or alter the states of devices [2, 5, 9]. The use of NLIs to fetch and return data from network performance databases by executing database queries largely remains unaddressed. Text-to-SQL translation models [14–19], which generate SQL queries based on natural language text inputs, can play a great role here by translating a user's natural language questions into desired SQL queries. However, these models cannot be directly used with speech inputs. Moreover, working with text transcribed from speech may lead to the introduction of noise. On the other hand, the contemporary text-to-SQL translation models usually assume that the database contains semantic names. The presence of the non-semantic table and column names, which can be often found in legacy applications, may pose a challenge to relate them with the natural language text.

Motivated by these aspects, in this work, we address the problem of querying the performance of WMNs using natural language. In particular, we develop the Natural Language Interface for Network Query (NLINQ), which allows querying the performance measurement data of a WMN using voice and text inputs. Such natural language questions are translated into SQL queries and executed in the database to return the results. To address the problem of non-semantic names in the database tables and columns—which can adversely affect the translation of text into SQL queries—we consider the use of database views with semantic names[1]. Moreover, we make domain-specific value corrections in the text (either original input or transcribed from speech) in order to improve the accuracy of the downstream text-to-SQL translation task. Figure 1 presents a high-level overview of NLINQ.

The novelty of this work is multi-fold. First, the voice-based, real-time interface offers a faster way to interact with and troubleshoot systems. It may be noted that the generation of SQL queries from speech inputs largely remains unexplored. Second, a hands-free approach can potentially improve the safety of work environments, in part, when network engineers install wireless routers atop long utility poles and need to query the status. Third, the proposed technique allows one to work with old, existing RDBMSs that use cryptic or non-semantic table and column names without requiring any modification to any table.

The specific contributions of this work are as follows:

- Generating and using logical database views with semantic names as proxies to the corresponding physical tables containing non-semantic table and column names.
- Fine-tuning Semi-autoregressive Bottom-up Semantic Parsing (SmBoP) [17], a state-of-the-art text-to-SQL

[1] By "semantic" name we mean a table or column descriptor that clearly conveys the meaning. For example, we say that "noise_5_ghz" is a semantic name for the column that stores 5 GHz noise values. In contrast, we say that "noise_1" is non-semantic, because it does not convey the meaning clearly and also because users would not query the "noise 1" value.

translation model, with networking domain-specific data in order to enable text-to-SQL translation for a WMN.
- Transcribing speech inputs into text and making domain-specific corrections on the transcribed text generated before sending for test-to-SQL translation, so that the downstream text-to-SQL translation task generates SQL queries with correct values.
- Designing the NLINQ prototype with support for text and voice inputs and evaluating the feasibility of real-time querying using a commercial, proprietary industrial WMN solution.

It may be noted that while this work specifically deals with industrial WMNs, the techniques presented herein are equally applicable to other large-scale systems, such as the Internet of Things (IoT) [20], Software-defined Networks [21], and Supervisory Control And Data Acquisition (SCADA) [22]. Moreover, the proposed model may also be potentially integrated with voice-based personal assistants to develop new, interesting applications, for example, querying the average humidity of a room in a smart-home system.

Figure 1 illustrates the architecture of NLINQ, together with different communication flows. When the NLINQ application server is launched, it authenticates with the speech-to-text translation server using a key (step 1). A JSON Web Token (JWT) is received in response, which is relayed to the client (Web browser). Subsequently, the client streams recorded audio along with the JWT to the speech-to-text server and receives the transcribed text in response. When the complete translated text is received by the client, it sends the text to the application server (step 6), which, in turn, relays the text to the text-to-SQL server along with a separate authentication key. The text-to-SQL server generates and sends back a SQL query based on the received natural language text. The received query is executed in the database connected to the Network Management System (NMS). Finally, the results of SQL query execution are sent back to the client.

The proposed architecture in Fig. 1 has a minimal dependency on the existing system—requiring only the database connection—and can potentially improve the experience of network engineers and business users trying to query the performance of the WMN. It may also be noted that many commercial speech-to-text transcription services offer edge-based solutions. Consequently, NLINQ can be potentially run entirely on-premise.

The scope of this work is limited to querying the performance of WMNs using data recorded in the database. In other words, any information present outside the said database cannot be queried using the proposed approach. Moreover, only Relational Database Management Systems (RDBMSs), which use SQL, are considered here. The assumption is justified by the fact that RDBMS remains the dominant choice of most professional developers [23]. Finally, the support for natural language is restricted to English only.

The paper is organized as follows: Section 2 presents a summary of related works. Section 3 discusses the problem statement and the solution approach. Section 4 presents a detailed discussion of the experimental setup and performance evaluation metrics. The experimental results are discussed in Section 5. Finally, Section 6 concludes this work.

2 Related Work

In this section, we present a short review of the use of natural language to manage network operations. Subsequently, we review some of the contemporary works on semantic querying of databases.

2.1 Natural language interfaces for networks and systems

Natural language interfaces, such as voice, text, and conversational assistants, in general, are becoming ubiquitous and finding a place in our daily lives, for example, in smartphones and dedicated physical devices, and are used across many domains.

Saha et al. [9] investigated the problem of intent-based management of network operations using natural language. In particular, when users express their intended actions using natural language, such as "create a flow between X and Y," programs or commands to achieve such actions are executed. Xu et al. [2] and Isyanto et al. [5], on the other hand, considered the use of natural language to manage IoT operations. However, contemporary natural language-based management solutions [5, 9] lack in addressing the problem of querying performance measurement data. In particular, such works seem to lack a focus on querying network performance based on historical data stored in a database. The latter scenario is different from executing a one-time command directly on a device, for example, to retrieve or set the current temperature or signal strength. Moreover, extending the architecture of such natural language-based management solutions [5, 9] to include the database querying functionality appears to be non-trivial.

To search a relational database that stores performance measurement data, one specifically requires to run SQL queries. NMSs and other similar management software usually provide a set of inbuilt reports to query such data using GUIs. However, a database may contain dozens of tables, where each table may have dozens of columns, which would require a generation of innumerable reports. On the other hand, each natural language question and SQL query may have different filtering criteria, which makes the problem further difficult. Finally, even if all such potential reports were

made available, users would need a tremendous amount of time to sift through them. This is undesirable in many scenarios, such as while troubleshooting a network problem, where one needs to quickly obtain information about the problematic nodes and gateways. In other words, querying a database using a set of pre-defined reports or programs (triggered based on intents) may soon become unmanageable. Consequently, there is a need to translate natural language questions into SQL queries on the fly to address these potential problems.

2.2 Semantic querying and text-to-SQL translation

Networks and NMSs use databases to store various types of data, such as packet-level measurements [24]. Wu et al. [25] considered the problem of querying network management databases (abbreviated as NEMA), which can contain several tables and a large number of records in each table. To this end, the authors proposed a series of algorithms that construct a graph database for efficient querying. In particular, the columns containing numerical values were matched by considering the difference in values and dot products. On the other hand, for matching the columns containing non-numeric values, the percentage of common records in two columns, as well as hashing techniques, were considered. The solutions were evaluated based on 21 tables containing millions of records. However, it may be noted that in such cases, the resulting graph databases constructed can become huge in size. On the other hand, although field-matching algorithms, in general, are novel, they perhaps lack in considering an important aspect: given any (correct) SQL query, an RDBMS already knows the optimal way to fetch the relevant records. This consideration has a significant practical impact. For example, Wu et al. reported an average computation time of about 15–47 minutes, which affects its suitability for any real-time, interactive application.

The translation of natural language text into SQL queries has gained considerable attention in the field of AI and NLP [15–18]. While the early approaches in the field largely leveraged grammar-based parsing of input texts and mapping them to SQL queries, contemporary approaches are based on deep learning [19]. The high-level objective here is to semantically relate tokens from natural language inputs to database schema—names of tables and columns, various keys, and possibly content—and the output SQL queries. Once such a relationship is approximated, during inference, output tokens representing table names, column names, and different conditions are generated forming an SQL query. For example, state-of-the-art NLP techniques, such as transformers, can be used to sequentially scan the input text and identify the potential table and column names. Modern sequence-to-sequence modeling approaches for text-to-SQL translation typically involve a decoder with constraints, for example, using an abstract syntax tree (AST), so that the output SQL queries are structurally correct in most cases [19]. The problem is somewhat different from generating programming language code from text because, apart from the query syntax, there are also schema-level constraints on the output. Text-to-SQL translation models are usually trained with large datasets, for example, Spider [26], a cross-domain dataset consisting of 200 databases, more than 10000 natural language questions, and more than 5500 unique SQL queries. However, generalizing the learning to generate SQL queries for unseen database schema constitute another challenge.

Herzig et al. [27] proposed a weakly supervised model for parsing tabular data based on Bidirectional Encoder Representations from Transformers (BERT) [28]. The proposed technique, however, requires passing the entire set of tabulated data as input, which may be unsuitable in real life for querying large tables. Lin et al. [14], on the other hand, proposed Bridge, where a hybrid input sequence of text, database fields, and contents was encoded using BERT. In order to address the relational ambiguity that may arise between text and database names, Wang et al. [15] designed RAT-SQL by considering schema encoding and schema linking. In particular, the authors jointly encoded the table names, column names, and words from the text as well as the relation graph among them. Rubin and Berant [17] designed SmBoP to translate natural language texts into SQL queries. SmBoP used RAT-SQL as the encoder. Unlike some prior works [15, 17] that used Long Short TermMemory (LSTM) decoders, SmBoP used a transformer decoder, which can capture contextual information in a better way. In particular, the decoder of SmBoP [17] used a bottom-up approach—in contrast to the widely used top-down approach—to build ASTs of target queries. The authors argued that the trees built using such an approach are more semantically relevant. Moreover, the training time was also found to reduce significantly.

The explosive growth of ChatGPT[2] has found applications in text-to-SQL translation as well. When ChatGPT is prompted with database structures and natural language questions, it generates relevant SQL queries[3]. The accuracy was found to be largely close to the state-of-the-art. The use of ChatGPT, however, has some shortcomings. First, it is required to provide database schema together with the natural language question, since the output SQL query is generated by semantically correlating the text with schema. Moreover, this must be done for every question that is asked. Therefore, when the database schema is large, which often can be the case with real-life systems, a large volume of (redundant) text must be transferred, since ChatGPT (and similar tools) are accessible only on the Internet. Given the fact that the database schema of a real-life product barely changes,

[2] https://chat.openai.com/

[3] https://github.com/THU-BPM/chatgpt-sql

such operations point to a certain degree of network inefficiency, in general, and may impact network latency. The second problem is the need to share detailed database schema with a third-party service. Recently, various organizations—and even a country—have put restrictions on the use of ChatGPT-like tools because of data privacy concerns [29, 30]. Consequently, the use of a ChatGPT-based text-to-SQL translation solution in the industrial context seems to be impractical at present.

It may be noted that although SmBoP [17] and similar text-to-SQL models are trained on large datasets, the adoption of such a model to query the performance measurement data of WMNs—or networks, in general—is challenging. First of all, text-to-SQL translation—including those based on large language models—inherently assumes that the names of the database tables and columns are semantic; non-semantic names may lead to incorrect mapping from texts to column names. The absence of semantic names, however, is often the case with real-life databases deployed in production environments, which have evolved over years and decades. Moreover, changing all such non-semantic names in an RDBMS is often impractical, since any such change would break all applications linked downstream.

The second challenge arises due to the domain-specific nature of the data involved. Many tables in databases store categorical values, such as WMN device type, which can be "Node" or "Gateway." Therefore, such values are case-sensitive and singular, and they must be corrected in the users' input texts if required, in order to prevent the generation of SQL queries with wrong value comparison.

On the other hand, database tables often store date-time values as UNIX epochs. In contrast, users specify dates in terms of year, month, and day. Consequently, such input types need to be identified and converted. Moreover, the syntax of certain SQL queries varies with RDBMSs and needs to be adapted.

Finally, text-to-SQL models translate text—and not voice—into SQL queries. Transcribing speech into text can lead to the introduction of additional noise. For example, a speech-to-text model may wrongly transcribe an IP address as "10.0 dot 0.1"—which will affect the correctness of the resulting SQL query. Therefore, wrongly transcribed IP addresses from speech inputs need to be corrected. It may be noted that the case sensitivity and exact spellings of some inputs may render conventional pre-processing approaches, such as stemming and lemmatization, ineffective. For example, depending upon how a user pauses while speaking, "Gateway" may be transcribed as "gate way"—with two words and a space in between—which indicates something very different.

To synthesize, we observe that although natural language-based network and systems management has been extensively investigated, its complimentary problem—querying operational performance data stored in databases using natural language—largely remains unexplored. On the other hand, the adoption of contemporary text-to-SQL solutions (or the development of a voice-to-text-to-SQL pipeline) has several practical challenges, such as non-semantic names and incorrectly transcribed inputs. Accordingly, in this work, we address some of these shortcomings to enable voice-based querying of a WMN's performance measurement database.

3 Problem description and solution

Let R be a relational database with a set T of tables (or table names). Let C_i be the set of columns (or column names) of table $T_i \in T$, $\forall i$, so that $C = \bigcup_i C_i$. It is assumed that all or most of the names in T and C are non-semantic. Further, let D be a set of natural language questions-SQL query pairs (henceforth referred to as dataset), where the SQL queries refer to the names in T and C.

Let $\mathcal{S}$ be a speech input. Let $\mathcal{T}_\mathcal{S}$ be the corresponding natural language text transcribed from the speech. Moreover, let $\tilde{\mathcal{T}}$ be a transformation of $\mathcal{T}$. Further, let Q be the SQL query corresponding to the text $\tilde{\mathcal{T}}$. The query Q refers only to the names in T and C.

Therefore, given a text-to-SQL translation model M_R trained with a dataset D and any speech input $\mathcal{S}$, our objective is to generate a SQL query $\hat{Q}$, which is the same, equivalent, or "close" to Q. Symbolically,

$$\hat{Q} \stackrel{\text{def}}{=} M_R(\tilde{\mathcal{T}}_\mathcal{S}). \tag{1}$$

In other words, (1) defines the pipeline for voice-to-text-to-SQL translation. Accordingly, the problem of voice-based performance querying of a network database can be divided into the following steps:

(S1) Create a mapping of T and C into semantic names.
(S2) Train (fine-tune) the model M_R based on the aforementioned mapping.
(S3) Transform text transcribed from speech by making appropriate corrections.
(S4) Run inference using M_R and the transformed text $\tilde{\mathcal{T}}_\mathcal{S}$.

The interpretation of close and equivalent queries would be presented in detail in a later section when we discuss the experimental results.

Figure 2 illustrates the steps of training a text-to-SQL translation model, together with the use of such a pre-trained model for real-time inference. The individual components of Fig. 2 are discussed in the remainder of this section.

3.1 Semantic database views creation

As noted in Section 2, changing table and column names in a production environment may be infeasible, in general. In order to alleviate this issue, we consider the use of database views, which are logical tables without any physical existence. A database view can be created by selecting columns from one or more tables. Moreover, the selected columns can be assigned new, semantic names.

In particular, let V_{T_i} be a database view corresponding to table T_i. V_{T_i} contains all the relevant columns from T_i. However, each such column is assigned a semantic name. Accordingly, we refer to V_{T_i} as a semantic view. It may be noted that V_{T_i} may contain additional logical columns that are not originally present in T_i but are useful in practice. For example, an original, physical table may contain two columns to store the signal and noise values in decibels. The corresponding semantic view may introduce an additional column, snr, to retrieve the signal-to-noise ratio value as the difference between the signal and noise columns. As a result, when the semantic views are created for each table in T, all concerned entities have semantic names, and therefore, a more accurate mapping of natural language text to column names can be obtained.

3.2 Dataset update

We assume that a dataset D of natural language questions and their corresponding SQL queries is available. Moreover, we assume that the SQL queries in D refer to the original, non-semantic table and column names. In general, it is the expected scenario when one plans to add a natural language-based querying feature to an already existing database. However, once the semantic views are created, those names in the SQL queries are replaced with new, semantic table and column names.

Algorithm 1 captures the steps for semantic view creation and the subsequent update of the dataset. The algorithm assumes that the semantic names for the original names are available in the form of N, for example, created by a database administrator. The outputs of Algorithm 1 are used to train or fine-tune a text-to-SQL model and generate M_R. At this point, steps (S1) and (S2) are completed. Moreover, all queries generated during the inference will refer to the names in V_T (step (S4)). It may be noted that typically, $|D| \gg |T| \times \sum_i |C_i|$, so the time complexity of the algorithm is bounded by $O(|D|)$.

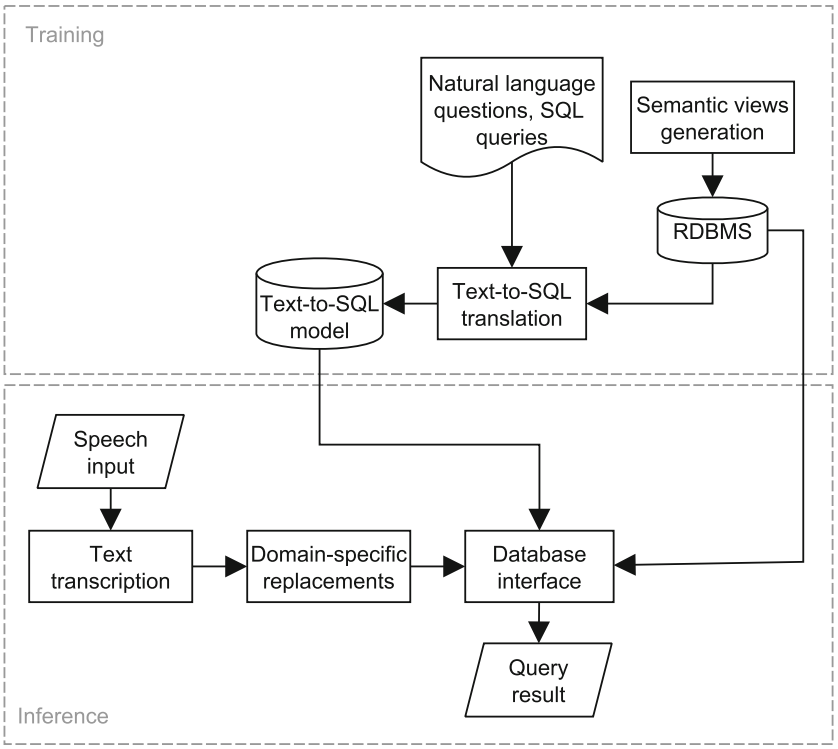

Fig. 2 Training a text-to-SQL translation model using natural language questions and the corresponding SQL queries, which refer to semantic names. The saved text-to-SQL translation model is later used for real-time inference, which uses text transcribed from speech inputs

Algorithm 1: Semantic views creation in the database and dataset update

Input:
- T: Non-semantic table names
- C: Non-semantic column names
- N: Semantic names for tables and columns
- D: Training dataset

Output:
- V_T: Semantic views
- D: Modified dataset referring to semantic views

$V_T \leftarrow [], M \leftarrow \{\}$ `// Empty list and hash map`
for each table $T_i \in T$ **do**
  $V_{T_i} \leftarrow$ Semantic view name from N correponding to T_i
  $M[T_i] \leftarrow V_{T_i}$
  for each column $c \in C_i$ **do**
    $K_i \leftarrow$ Semantic view name from N correponding to C_i
    $M[C_i] \leftarrow K_i$
    Append K_i, C_i to V_{T_i} `// Both the original and the new column names are required`
  end for
  Append V_{T_i} to V_T
end for
for each view $V \in V_T$ **do**
  Execute SQL statement to create V
end for
for each datapoint $d \in D$ **do**
  $q \leftarrow$ the SQL query of d
  Replace T_i with $M[T_i]$ and C_i with $M[C_i]$ in q
end for

3.3 Domain-specific patterns replacement

As discussed in Section 2, natural language text or voice inputs of users may need some pre-processing to improve the correctness of the SQL queries generated, which constitutes step (S3). It may be noted that this step is required only during the inference stage since inconsistencies can be largely avoided while generating the training data under human supervision.

Algorithm 2 illustrates the domain-specific corrections of natural language texts at a high level. In the first step, zero or more regular expressions are applied to correct potentially wrongly transcribed network entities from users' speech. In this work, we consider regular expressions related to IP addresses only. However, this step is applicable to many other scenarios, in general. For example, correcting the transcription of the MAC address of a network interface that consists of the colon symbol.

In the next step, the database columns with categorical values are considered. Any incorrect variation of such values is changed to the corresponding correct value. As discussed in Section 2.2, a WMN database may have fields that store values indicating certain types, such as "Node" and "Gateway." Let us consider the question, "what is the average 5.8 gigahertz snr of nodes?" The corresponding SQL query is SELECT AVG(v_performance.snr_58) FROM v_performance WHERE v_performance.type = 'Node'. However, since text-to-SQL translation schemes typically copy values from natural language texts, the generated query would be SELECT AVG(v_performance.snr_58) FROM v_performance WHERE v_performance.type = 'nodes'. The resulting query would fail to find any match in the database because of the incorrect spelling. Therefore, in order to avoid copying wrong values, Algorithm 2 edits input texts by replacing the variations of known categorical values with the actual values stored in the database. Finally, all human-readable date-time values are replaced with the corresponding UNIX epoch. For example, the string "Monday, July 11, 2022 11:00:00 AM" in any input text would be replaced with "1657537200." The output of the algorithm, $\tilde{\mathcal{T}}$, is used to run inference using the text-to-SQL translation model. Finally, the resulting SQL query is executed in the database to fetch the desired data.

Algorithm 2: Domain-specific corrections performed on natural language text inputs

Input:
- $\mathcal{T}$: Natural language text of a question
- ρ: Regular expressions and replacement patterns
- κ: Categorical values and their variations

Output:
- $\tilde{\mathcal{T}}$: Transformed natural language text

$\tilde{\mathcal{T}} \leftarrow \mathcal{T}$
for each pattern $p \in \rho$ **do**
  if p matches $\tilde{\mathcal{T}}$ **then** $\tilde{\mathcal{T}} \leftarrow$ Apply replacement of p to $\tilde{\mathcal{T}}$
end for
for each categorical value and its variations $k \in \kappa$ **do**
  if any variation of k found in $\tilde{\mathcal{T}}$ **then** $\tilde{\mathcal{T}} \leftarrow$ Replace variation with k in $\tilde{\mathcal{T}}$
end for
if $\tilde{\mathcal{T}}$ has any date-time pattern **then** $\tilde{\mathcal{T}} \leftarrow$ Update text by replacing date-time value with UNIX epoch

In general, the number of regular expressions required and the number of columns with categorical values are usually small. Moreover, the average length of $\mathcal{T}$ is typically short. Consequently, the average time complexity of Algorithm 2, which essentially processes a fixed-length text, is in the order of $O(|\mathcal{T}|)$. In other words, step (S4) executes fast, without significantly affecting the overall processing time.

4 Experimental setup

In this section, we discuss the data preparation and prototyping environment. We also discuss the different metrics used for performance evaluation.

4.1 Data and environment

We developed the NLINQ prototype to query the network performance database of a commercial, proprietary WMN solution. The mesh network uses the 2.4 GHz, 4.9 Ghz, and 5.8 GHz frequency bands to communicate. The nodes and gateways of the WMN periodically send their measurement data, such as latency, signal strength, and throughput, to the NMS, which inserts those records into a relational database. Our objective was to query said database using natural language. We considered three tables from the database, which collectively consist of 95 columns. Three semantic views, one for each physical table, were created by considering a subset of 51 columns. In addition, three more logical columns were added to one of the semantic views, so the total columns count under consideration became 54.

Following the Spider [26] format, we manually created a dataset based on the contents of the NMS database to fine-tune SmBoP for our specific use case. In particular, the dataset consisted of 500 pairs of natural language questions and their corresponding SQL queries. The queries referred to the semantic views and their columns. We performed a 50:30:20 split on the dataset to generate the training, validation (dev), and test sets[4]. Based on GPU memory usage, the training batch size was set to 18. The best result was obtained after 183 epochs.

Moreover, Spider [26] categorizes any SQL query into four types based on "hardness": easy, medium, hard, and extra hard. For example, SQL queries that have only filtering or ordering clauses, such as WHERE and LIMIT, are deemed to be easy. Similarly, other SQL components, such as set and aggregation operations, are considered to categorize into the other three types. Accordingly, Fig. 3 summarizes the query hardness for our test dataset. In particular, most of our queries were deemed to be extra hard, whereas none of them were in the hard category.

We developed NLINQ as a Web application using the Flask framework for Python 3.7, as illustrated in Fig. 1. The NLINQ application server, which is served using the Waitress[5] server, and the SmBoP text-to-SQL translation server were deployed independently on a laptop and a cloud virtual machine (VM), respectively. In particular, we used a VM instance with Tesla T4 GPU on Amazon Web Services[6]. The same VM was used to run the fine-tuning of the SmBoP model. Table 1 presents an overview of the deployment environments. The NLINQ application server used SocketIO[7] for bidirectional communication with the clients. We used the pyduckling-native[8] to translate any date-time value present in the text into UNIX epochs. In addition, we also used the speech-to-text translation service provided by Azure Cognitive Services[9] for real-time transcription of users' speech inputs into text. Figure 4 presents a screenshot of the NLINQ prototype.

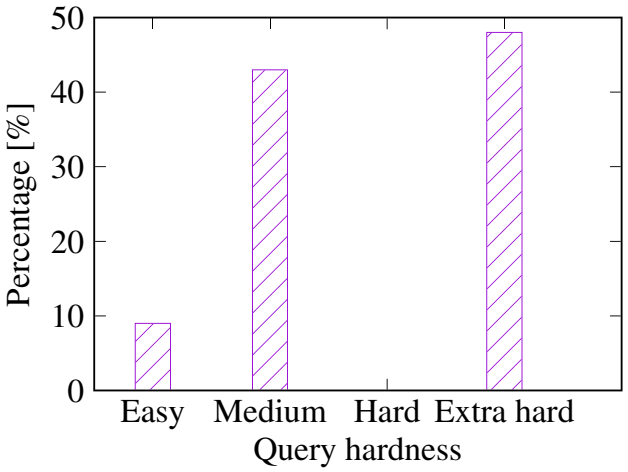

Fig. 3 SQL query hardness for the test dataset. The difficulty level of about half the SQL queries was "extra hard." These often included a sub-query to select the last recorded timestamp from the tables

We also implemented an alternative version of NLINQ, which continuously listens to the ambient sounds, and where the speech-to-text translation service is triggered on detecting a custom wake word. Therefore, the prototype enables an entirely hands-free mode of operation. Since the rest of the functionality remains the same, we do not discuss this alternative version further in this work.

We also used Gensim[10] to identify the semantic similarities among the column names from the semantic views. In particular, we used Gensim to obtain the vector representations of the word phrases, based on which the cosine similarities were computed. The value ranges between −1

[4] Finegan-Dollak et al. [31] proposed question-based and query-based splits of text-to-SQL translation datasets. The former splits the entire dataset into train, validation, and test sets based solely on the natural language questions. This, however, may lead the presence of similar SQL queries in different data splits. Query-based split, on the other hand, ensures that similar queries are part only one split set. It may be noted that, in this work, we consider an already pre-trained text-to-SQL translation model and fine-tune it with WMN domain-specific data. Since we are considering a specific domain, the number of potential queries and their variations constitute a relatively small subset. In addition, there are certain queries that users execute more often than the others. The natural language expressions for such frequent queries may widely vary, although the corresponding SQL queries may look similar. Therefore, it is relatively more important to correctly generate SQL queries for the frequently encountered questions. Consequently, we consider the question-based split of the dataset

[5] https://pypi.org/project/waitress/

[6] https://aws.amazon.com/

[7] https://socket.io/

[8] https://pypi.org/project/pyduckling-native/

[9] https://azure.microsoft.com/en-us/services/cognitive-services/speech-to-text/

[10] https://radimrehurek.com/gensim/

Table 1 Overview of the deployment environments

	NLINQ Application Server	SmBoP Inference Server
Type	Laptop	Cloud VM
OS	Windows 10	Ubuntu 18.04
CPU	Intel(R) Core(TM) i5-10310U CPU @ 1.70GHz 2.21 GHz	4x Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
RAM	16 GB	16 GB
GPU	N/A	1x NVIDIA Tesla T4 with 16 GB memory

(strongly opposite) to 0 (uncorrelated) and 1 (very similar). The objective of this step was to investigate the extent to which semantically similar column names affected the query generation accuracy. In other words, we wanted to qualitatively identify whether or not the partial similarity among the column names of the views affected the correctness of the SQL query generated.

For a comparative assessment, we considered Bridge [14], another state-of-the-art text-to-SQL translation model, as a benchmark. In particular, we ran inference using a Bridge model pre-trained on the Spider dataset. In other words, this model has never seen any WMN domain-specific data. For the purpose of inference, i.e., generating SQL queries from text, we considered the database with semantic view and column names. Separately, we also fine-tuned the pre-trained Bridge model with WMN domain-specific data and then ran inference. We measured the translation accuracy and the average computation time for both scenarios.

4.2 Evaluation metrics

We considered the following metrics to evaluate the performance of text-to-SQL translation:

- Exact matching [26]: An exact match occurs when all the components of the predicted query $\hat{Q}$ are the same as that of the gold (correct) query Q. The value of this metric should be close to 100%. Here, "components" refer to an abstract representation of a SQL query, such as the column names present, the aggregation operators used, and so on.
- Execution accuracy [26]: In this case, the exact values (based on which rows are to be filtered) are taken into account and compared with the values from the corresponding gold queries. The value of this metric should be close to 100%, which is an indication that most SQL queries are correctly generated. To illustrate, let us

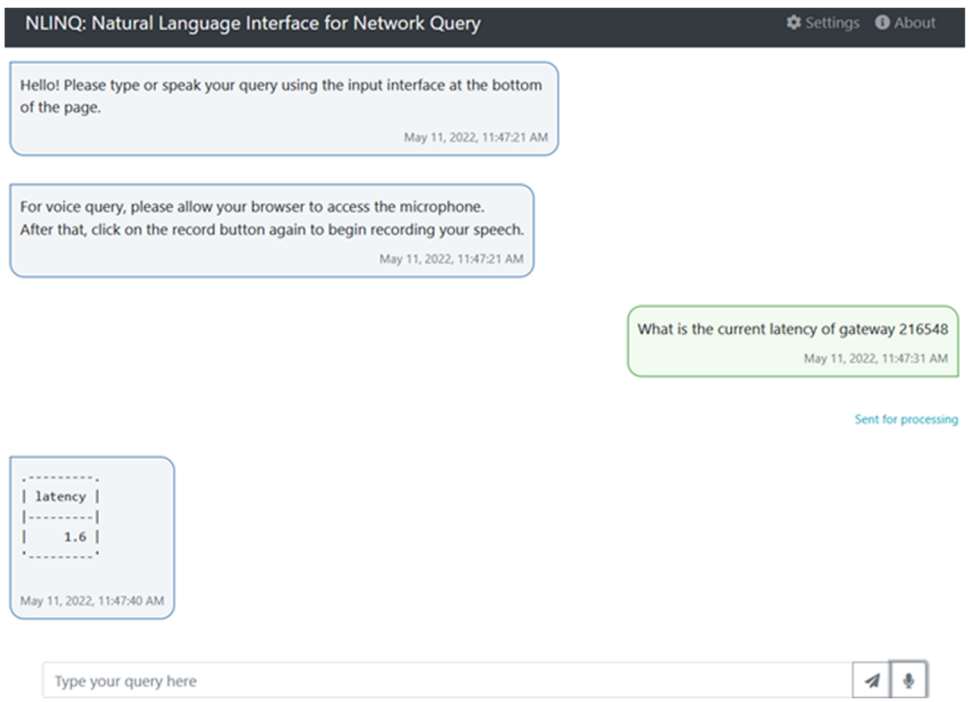

Fig. 4 A screenshot of the NLINQ prototype. User inputs—originally as text or transcribed from speech—are displayed inside text bubbles aligned to the right side. System-generated messages are aligned to the left. Each message shows the timestamp when it was generated

Table 2 Sample natural language questions and the corresponding SQL queries using semantic names generated from the original column and table names

#	Question	SQL Query
Q1	what is the current latency of gateway 216548?	SELECT v_gateways.latency FROM v_gateways WHERE v_gateways.id = '216548' AND v_gateways.timestamp = (SELECT MAX(v_gateways.timestamp) FROM v_gateways)
Q2	what is the historical IP address, latency, upstream throughput, and downstream throughput of gateway 216548?	SELECT v_gateways.timestamp , v_gateways.ip_address, v_gateways.latency, v_gateways.upstream_throughput , v_gateways.downstream_throughput FROM v_gateways WHERE v_gateways.id = '216548'
Q3	what was the latency of gateway 216548 on August 19, 2021, at 9:00 AM?	SELECT v_gateways.latency FROM v_gateways WHERE v_gateways.id = '216548' AND v_gateways.timestamp = 1629363600
Q4	what was the IP address, latency, upstream throughput, and downstream throughput of gateway 216548 at timestamp 1629295200?	SELECT v_gateways.ip_address , v_gateways .latency, v_gateways.upstream_throughput , v_gateways.downstream_throughput FROM v_gateways WHERE v_gateways.timestamp = 1629295200 AND v_gateways.id = '216548'
Q5	Show me the average 5.8 gigahertz snr for each node IP address.	SELECT v_performance.ip_address , AVG(v_performance.snr_58) FROM v_performance WHERE v_performance.type = 'Node' GROUP BY v_performance.ip_address

consider a gold query, SELECT AVG(latency) FROM v_gateways WHERE id = '216548'. Let us consider that the predicted query for the corresponding question is: SELECT AVG(latency) FROM v_gateways WHERE id = ' '. In this case, the gold and predicted SQL queries have the same structural components, which results in an exact match. However, the values used by the queries are different—216548 versus blank—which produces different results sets. Therefore, this particular predicted query does not contribute toward execution accuracy.

- Text-to-SQL translation time: It is measured as the average time taken to translate a natural language text into a SQL query.
- Turnaround time: It is the time duration from the moment a user stops recording speech by clicking on the stop button until the moment when the result of the SQL query execution is received by the Web browser. For text inputs, the turnaround time calculation begins from the moment when a user has completed typing in the text and clicked the submit button. Since this metric is an indication of how interactive an application is, the value should be low, for example, not more than 5 seconds.
- Response time of the text-to-SQL translation server: We measure the average time taken by the text-to-SQL server to process an incoming request and send the response back. In other words, the average response time includes the time taken to generate a SQL query based on an input text (i.e., the translation time), as well as the network round-trip time. It may be recalled that the text-to-SQL translation server only generates the SQL queries, it does not execute those queries. Therefore, the response time is marginally less than the turnaround time, since the latter also includes the time taken to execute a query in the database.
- Requests processed per second: It is measured as the number of requests processed by the text-to-SQL translation server per second, on average. A higher value of this metric enables a higher degree of concurrency.

These last two metrics provide useful insights while conducting load testing.

In order to evaluate the run-time performance, we considered a set of natural language questions, as shown in Table 2. The corresponding SQL queries are also presented. The set presented in the table, in general, is representative of different kinds of questions (queries) used to fine-tune SmBoP. Some of these queries are simple—select a single column and apply a single filter, whereas some others are relatively hard—select several columns and apply multiple filters. Moreover, the natural language texts vary in length. Our objective was to identify how different questions affected the text-to-SQL translation time, if at all. We provided these natural language questions as text inputs, 10 times each, and measured the average turnaround time.

We used Locust[11] to perform load testing of the text-to-SQL translation functionality. In particular, separate tasks were defined to generate Web requests in order to translate each natural language question. Each task had an equal likelihood of execution and ran for a period of 3 minutes. In each iteration, the number of users was constant. The number of users concurrently generating requests varied from 1 to 21 in steps of 5 in different iterations. A pause time of 10–15 seconds was introduced between any two successive tasks (i.e., generation of translation requests) in order to mimic the scenario where users look at the results of queries. The average response time for each request was measured.

[11] https://locust.io/

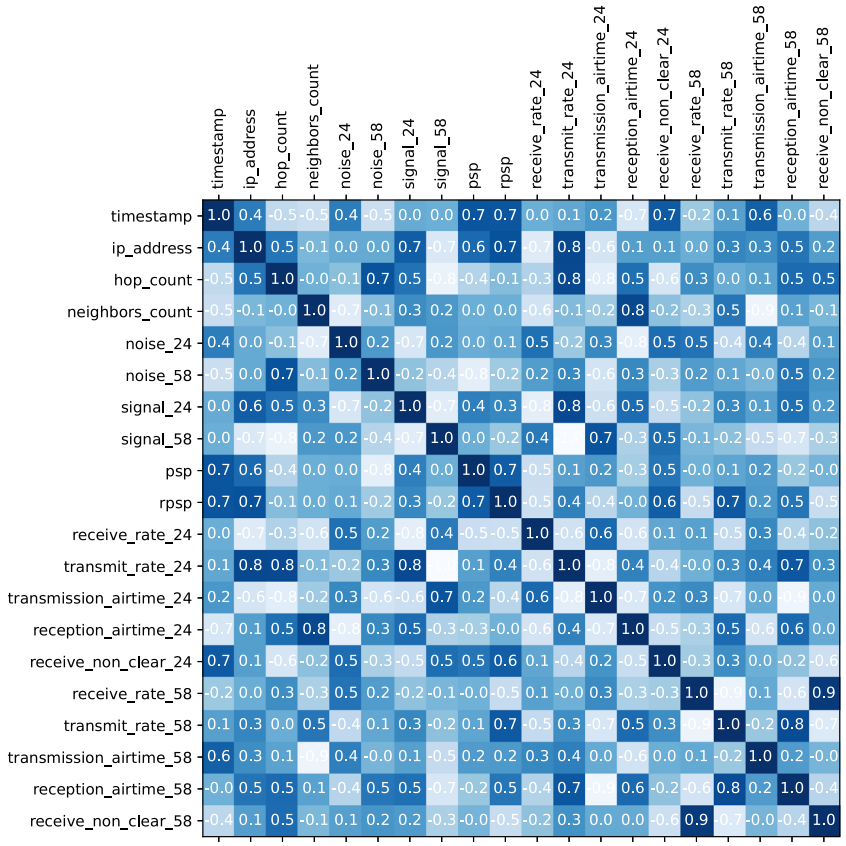

Fig. 5 Similarity among the semantic column names generated from original, non-semantic names. The figure, in general, is representative of real life, where all the names are not expected to be semantically distinct. Therefore, the evaluation of text-to-SQL translation schemes, in a sense, is also the evaluation of their performance in the presence of partially similar column names

5 Results

In this section, we take a detailed look at the experimental results. Subsequently, we discuss the pros and cons of NLINQ.

5.1 Query performance

We begin by considering the cosine similarities of a subset of the semantic column names, as shown in Fig. 5. In the ideal scenario, all names should be different, which would result in a cosine similarity value of zero. However, the positive values in the figure indicate varying degrees of correlation. For instance, the name "rpsp" appears to be quite similar to "timestamp" and "ip address." On the other hand, the names "hop count" and "transmit rate 58" appear to be distant. In other words, it is clear that, in real life, we can expect a certain degree of similarity, on average, among the different column names. At this point, Fig. 5 by itself does not offer any further insight. Therefore, we now look at the correctness of query generation, keeping in mind the partial similarity among the semantic column names.

Table 3 presents the accuracy of the fine-tuned SmBoP model for the dev and test datasets. In particular, in the dev set, the exact match was 92.663%. In other words, the structure of the predicted (or generated from natural language text) SQL queries matched with that of the target queries about 926 times out of 1000, on average. The execution accuracy, which considers the values used by the query, was 91.988%. We obtained a very close performance with the test set as well, with exact match and execution accuracy, respectively, at 89.021% and 91.028%. In this context, it may be noted that the execution accuracy and the exact match accuracy of the original SmBoP model based on the Spider dataset are 75.3% and 74.6%, respectively. The results, in general, indicate that domain-specific fine-tuning of text-to-SQL models can be highly useful. These results should also be interpreted by taking Fig. 5 into consideration. In other words, even with a few semantically similar column names, SQL queries can be generated with significant accuracy. This observation qualitatively answers the question that we posed at the end of Section 4.1.

Table 3 Accuracy of text-to-SQL translation for WMNs using a fine-tuned SmBoP model

Metric	Dev %	Test %
Exact match	92.663	89.021
Execution accuracy	91.988	91.028

Figure 6 shows how the aforementioned performance metrics fared for queries with different hardness. In particular, the execution accuracy for the easy category was 100%, which means all the underlying values necessary for filtering data, such as device identifier and timestamp, were correctly recognized. The same for the extra hard category was about 91.666%. On the other hand, the exact match for all the categories was close to 89%. The results, in general, indicate the fine-tuned model can translate natural language texts into SQL queries of varying hardness levels with very high accuracy.

We now take a detailed look at the prediction errors in text-to-SQL generation using SmBoP. In other words, we are interested to know why incorrect SQL queries were generated, which led to the accuracies in Fig. 6 being less than 100%. Similar error categories have been considered by Finegan-Dollak et al. [31]. We, however, follow a slightly different nomenclature so that these errors can be easily interpreted by the users of NLINQ, who may not necessarily be experts in NLP. In particular, we manually investigated the concerned output SQL queries and classified each such error into the following five categories:

- Equivalent queries: Two SQL queries can be structurally different and yet functionally equivalent. For example, a filtering criterion say, latency between 2 and 4, can be specified using the SQL clause "latency >= 2 and latency <= 4" or "latency between 2 and 4." Finegan-Dollak et al. [31] called such errors as "logic variants."
- Queries fetching extra columns: The predicted SQL query fetches one or more extra columns, but the same rows as the target query.
- Missing column: The predicted SQL query does not fetch at least one column, as specified in the target query. However, the same rows are returned. The previous error and the current one are referred to as "entity problem only" by Finegan-Dollak et al. [31].
- Incorrect query: The predicted SQL query is incorrect because it, for example, applies wrong filtering criteria or uses wrong values. The resulting SQL query, however, is executable. This roughly relates to the "different template" category [31].
- Error: The final category consists of predicted SQL queries that result in execution errors, for example, because of the wrong syntax. Finegan-Dollak et al. [31] labeled the preceding two errors as "no template match."

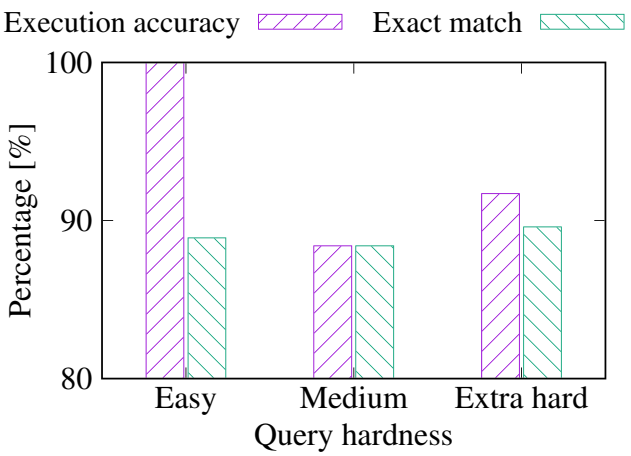

Fig. 6 Text-to-SQL translation accuracy for different query hardness

Table 4 Summary of text-to-SQL translation errors for WMNs using a fine-tuned SmBoP model

Error Category	Dev	Dev %	Test	Test %
Equivalent	0	0.000	1	9.091
Extra	2	18.182	1	9.091
Missing	5	45.455	2	18.182
Incorrect	4	26.363	6	54.545
Error	0	0.000	1	9.091

Based on the above categories, Table 4 summarizes the different types of prediction errors obtained using SmBoP. The individual counts and the corresponding percentages are displayed. For instance, in the test set, one of the prediction errors contained an equivalent SQL query. Another prediction error was caused because the SQL query generated selected an extra column. It may be noted that these two categories of prediction errors still produce the desired output. In other words, in the test set, about 18% prediction errors are effectively not errors, but correct output. Similarly, we find that about 45% of the prediction errors in the dev and 18% of the prediction errors in the test produced partially correct results. In other words, the effective accuracy of the fine-tuned SmBoP text-to-SQL translation model is marginally higher than as noted in Table 3.

In order to evaluate whether or not the use of semantic views with semantic column names provides any value, we also separately fine-tuned SmBoP with a dataset that used the original, physical, and non-semantic table and column names. In this case, we considered the same number of questions and the same natural language texts, as with the semantic names-based fine-tuned model. However, instead

Table 5 Accuracy of text-to-SQL translation for WMNs when SmBoP was fine-tuned using the original, non-semantic table and column names

Metric	Dev %	Test %
Exact match	6.344	4.155
Execution accuracy	7.049	5.223

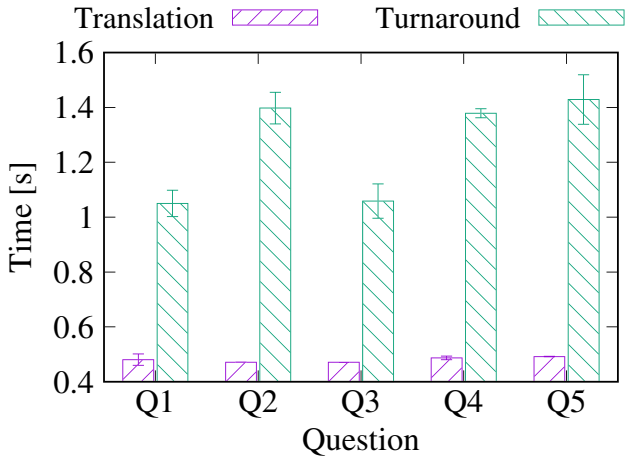

Fig. 7 Average translation and turnaround times for different questions. The text-to-SQL translation times for the questions were found to be largely similar

of the semantic column and view names, we used the original, non-semantic names from the physical tables. The corresponding results are displayed in Table 5. It can be observed that the accuracy was significantly poor. This is because, in most cases, the word phrases from natural language inputs could not be properly mapped to the original column names, which are often non-descriptive. In contrast, the use of semantic views as a proxy with semantic table and column names improved the text-to-SQL translation accuracy by about 13–21 times. Moreover, this significant improvement was achieved using a relatively small fine-tuning dataset.

Figure 7 shows the average time taken to process different natural language questions together with 95% confidence intervals. In particular, the average text-to-SQL translation time was in the range 0.471–0.491 seconds when the cloud VM with GPU was considered, as described in Table 1. However, on the physical system mentioned in the table, the average translation time was about 7.357 seconds. This underscores the need for faster CPU and GPU for text-to-SQL translation.

Although minor, the average translation times for Q4 and Q5 appeared to have statistically significant differences from those for Q2 and Q3, as indicated by their non-overlapping confidence intervals. If we look at the questions, Q4 requires an additional filtering criterion in the SQL query's WHERE clause. On the other hand, Q5 is different from the others and requires the use of an aggregation function in the resulting SQL query. These aspects may potentially explain the difference in time spent in translation.

The figure also shows the turnaround times measured in NLINQ when the inputs were in the form of natural language texts by a single user. In general, the turnaround times for questions Q1 and Q3 were less than that of the others. A possible explanation is that the output returned for these questions was only a single value, in contrast to several rows and columns of data.

When all the questions are considered together, the average turnaround time for text inputs becomes 1.263 seconds. Since the translation time is less than 0.500 seconds and SQL query execution takes a few milliseconds, the bulk of the turnaround time is accounted for by the round trips to the NLINQ applications server and text-to-SQL translation server. In addition, it may be noted here that, for speech input, we introduced a waiting time of 0.750 seconds after the recording was stopped. This is to ensure that the entire transcribed text from the speech was received before the text was sent for translation. Accordingly, the average turnaround time for speech inputs becomes 2.013 seconds. The waiting threshold for speech inputs can be changed based on network latency.

It may be recalled from Section 2 that NEMA [25] constructs a graph database in order to enable the semantic querying of network databases. Since NEMA and NLINQ take entirely different approaches and evaluate using different datasets and infrastructure, these two methods are not directly comparable. Keeping this caveat in mind, in Table 6, we take a comparative look between our experimental results and the observations originally reported for NEMA by the concerned authors [25]. Table 6 shows that the accuracy of NLINQ and NEMA, when evaluated using their respective datasets, are very close. In other words, both schemes can generate correct SQL queries almost all the time for the respective datasets concerned. However, the time taken for computation seems to be largely different. As noted in the preceding discussion, when a VM with GPU is used, SmBoP can translate a text into an SQL query in about 0.471 seconds. The almost insignificant computation time is a key enabler for the real-time use of NLINQ.

Table 6 Comparative performance of NLINQ, using both semantic and non-semantic names, with similar solutions

	SmBoP fine-tuned with original column and table names	NLINQ (fine-tuned SmBoP) with semantic names	Bridge (pre-trained on Spider dataset) [14]	Bridge fine-tuned with semantic column and table names	NEMA [25]
Accuracy %	7.049	91.028	4.999	64.000	≈ 95
Computation time [s]	0.471	0.471–7.357	1.512	3.833	≈ 939–2832

Table 6 also compares the performance of Bridge and SmBoP. When Bridge [14], pre-trained on Spider data, was used to generate SQL queries—without any fine-tuning—for the WMN domain-specific questions, the resulting accuracy was about 4.999%. In this case, the semantic table and column names from the semantic views were used. In addition, experimental results revealed that Bridge failed to generate a SQL query, i.e., generated empty strings, for 42% of the natural language questions. However, when Bridge was fine-tuned using WMN domain-specific data consisting of the semantic table and column names, the exact match and execution accuracy measures were found to be 64.000% and 63.000%, respectively. Moreover, the fine-tuned Bridge model failed to generate a SQL query for only 1% of the questions. However, experimental results also revealed that the fine-tuned Bridge model generated relatively more erroneous SQL queries than the fine-tuned SmBoP model (three versus one). On the other hand, the average inference time of Bridge (using GPU) was found to be significantly higher than that of SmBoP. In particular, the fine-tuned Bridge model generated the correct SQL query for Q1, which consisted of a subquery, in about 6.751 seconds. This might have been due to the combined effects of large beam size, schema-consistency check, and heuristics used by the decoder, which resulted in the discarding of a large number of wrong queries before the correct one was found. In contrast, the pre-trained Bridge model generated a wrong query in about 0.612 seconds. It may be noted that the fine-tuned model took relatively much lower time to generate (correct) SQL queries for questions Q2 through Q5, but they were still higher than the average inference time of SmBoP. Overall, the generally slow response of Bridge's decoder largely indicated its unsuitability for real-time applications.

5.2 Computational aspects

Figure 8 plots the average response time of the text-to-SQL translation server versus the number of concurrent users. As expected, the response time, in general, increased when the number of users submitting text-to-SQL translation requests increased. The line plot in Fig. 9 indicates the number of requests created per second (y-axis on the right side) by these users (x-axis). Figure 8, in general, indicates that the average response time for Q3 was the least. From Table 2 it may be noted that although the length of Q3 is slightly longer than that of Q1, the SQL query generated for the former is shorter. This could be a possible reason behind Q3, with similar or shorter payload length, having relatively lower response time, in general. Moreover, when multiple requests were submitted concurrently, the multithreading capacity of the Web server—in our case, Waitress—also affected the response time. In particular, with 21 concurrent users, which resulted in 1.294 requests created per second, most of the response times were about 3 seconds or more. This is an indicator that, even if the Web server can handle all these requests, the turnaround times would get much higher.

Figure 9 plots the requests processed per unit time (y-axis on the left side). This metric increased with time because, when the number of concurrent requests increased, the Web server also increased the utilization of allocated resources. Finally, we note that the results of load testing beyond the depicted levels indicated a performance degradation, with the VM becoming non-responsive. Therefore, the VM configuration in Table 1 can be useful to serve up to about 20 concurrent users with about one request created per second, on average. Since NLINQ is expected to be used by a relatively small team of network engineers, the specified configurations and performance should be at a satisfactory level. However, if a still higher degree of concurrency is required, a

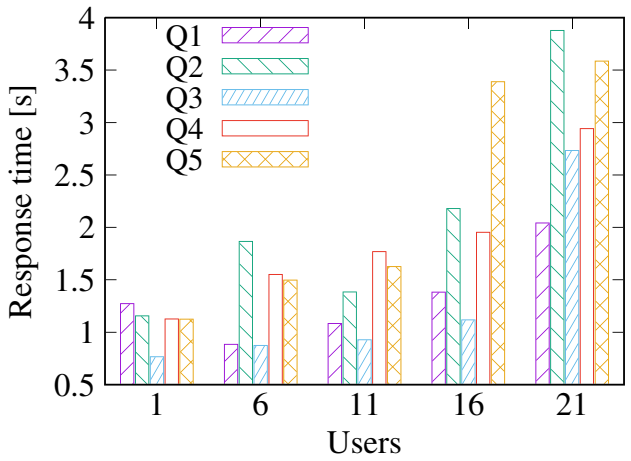

Fig. 8 The response time of the text-to-SQL translation server increased with the number of concurrent users

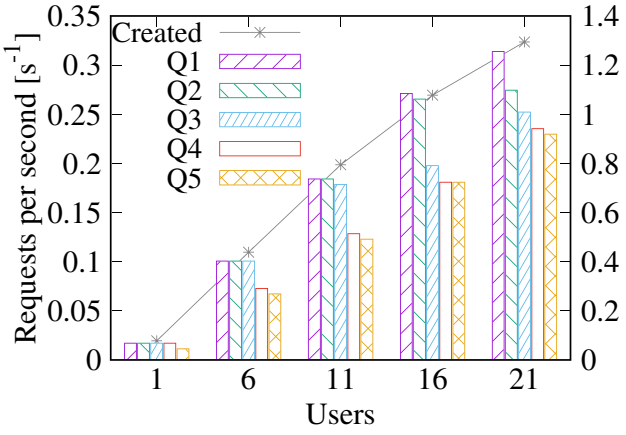

Fig. 9 The number of requests served (and created) per second increased up to a threshold of about 21 concurrent users because the Web server also increased its resource utilization in parallel

VM with more resources or additional VMs configured with load-balancing capability may be considered.

5.3 Discussion

We conclude this section by taking a holistic look at NLINQ's features and performance. Overall, NLINQ offers several advantages, such as:

- The ability to fetch results from a database using a hands-free approach with voice inputs. This can be highly beneficial for field and installation engineers, among others. This is made possible by a high degree of accuracy and a low turnaround time.
- Since NLINQ fine-tunes a state-of-the-art text-to-SQL translation with WMN domain-specific data, the training time and expenses are significantly low. In other words, customers can deploy this solution with relative ease.
- A significant advantage of NLINQ is that it works even with databases having non-semantic names by creating semantic views. Database view creation is a relatively simple operation and does not affect the database in any adverse way. This provides confidence to customers to deploy the solution.

On the other hand, there are certain areas where NLINQ can do better, for example:

- NLINQ runs queries against a relational database only. However, different non-relational and time series databases are increasingly being used today, and their query language can be different from SQL.
- NLINQ does not indicate or explain the error in case a wrong SQL query is generated. However, this falls outside the scope of text-to-SQL translation—it is typically left to the users to evaluate query correctness post its generation.
- NLINQ remains to be tested with more complex queries. However, whether or not such complex structures arise in the context of performance measurement databases needs to be considered as well.

6 Conclusion

Monitoring and management of networks and systems frequently involve querying their performance measurement data stored in databases, for example, to identify the nodes with low throughput and to confirm running status. Contemporary solutions often involve complex GUIs and CLIs, whose use may require time, skill, and cognitive load. The recent advances in AI and NLP can improve the contemporary approach toward network management and monitoring, which would, in turn, also improve customer experience. To this end, in this work, we investigated the problem of querying the performance of WMNs using natural language, where periodic performance monitoring data are stored in an RDBMS. In particular, we addressed the problem where questions are asked using natural language—as speech or text—based on which appropriate SQL queries are generated to query the network database. We fine-tuned SmBoP, a state-of-the-art text-to-SQL translation model, with our database-specific data. The fine-tuned model generated SQL queries with up to about 92% accuracy, which indicates its suitability for real-life use. The proposed methodology leverages database views created based on the physical tables. Therefore, it can be used with any relational database even if they have non-semantic table and column names. In other words, NLINQ provides strong backward compatibility with existing and legacy RDBMSs. We verified the feasibility by developing the NLINQ prototype, which enables seamless integration with any NMS and allows querying its database records using real-time speech and text. This was supported by the fact that the end-to-end turnaround time of query-response was about 1–2 seconds, even when the solution was deployed on the cloud and not on-premises.

This work can be extended in the future in different ways. Currently, NLINQ allows only querying what is stored in an RDBMS. However, not every device parameter measurement may be available there. Consequently, an intent-based approach to query the concerned devices directly may be considered. In fact, combining intent-based network management [9] with natural language-based querying would help to realize the full potential of natural language interfaces. On the other hand, this work can also be extended by considering a context-based querying approach. In such scenarios, unlike what is presently achieved using SmBoP, the query and its response in a certain step would depend on one or more queries asked immediately before. This would allow users to ask their questions more interactively and refine their questions as required. To this end, contextual text-to-SQL models [32, 33] may be considered. Finally, the size and diversity of the dataset may be increased to further improve the accuracy. In this context, a query-based split [31] of the dataset for fine-tuning may also be considered.

Acknowledgements Not Applicable

Author Contributions All the authors contributed equally to this work.

Funding Not Applicable

Data Availability Data are collected from a proprietary system and are unavailable.

Declarations

Competing interests The authors have made the following first filing: Barun Kumar Saha, Paul Gordon, and Tore Gillbrand, "Voice-based Performance Query with Non-semantic Databases," USA Patent Application No. 17/950,956, Sep. 22, 2022.

Ethical approval Not Applicable

References

1. Liu H, Zheng C, Li D, Shen X, Lin K, Wang J, Zhang Z, Zhang Z, Xiong NN (2022) EDMF: Efficient Deep Matrix Factorization With Review Feature Learning for Industrial Recommender System. IEEE Trans Industrial Informat 18(7):4361–4371
2. Xu Z, Wu H, Chen X, Wang Y, Yue Z (2022) Building a Natural Language Query and Control Interface for IoT Platforms. IEEE Access 10:68655–68668
3. Nassiri K, Akhloufi M (2022) Transformer models used for text-based question answering systems. Appl Intell. https://doi.org/10.1007/s10489-022-04052-8
4. Narechania A, Srinivasan A, Stasko J (2021) NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization from Natural Language Queries. IEEE Trans Visual Comput Graphics 27(2):369–379
5. Isyanto H, Arifin AS, Suryanegara M (2020) Performance of Smart Personal Assistant Applications Based on Speech Recognition Technology using IoT-based Voice Commands. In: 2020 International conference on information and communication technology convergence (ICTC), pp 640–645
6. Li Y, He J, Zhou X, Zhang Y, Baldridge J (2020) Mapping Natural Language Instructions to Mobile UI Action Sequences. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, Online, pp. 8198–8210
7. Ahkouk K, Machkour M (2023) SQLSketch-TVC: Type, value and compatibility based approach for SQL queries. Appl Intell 53(4):3889–3898. https://doi.org/10.1007/s10489-022-03587-0
8. Saha BK, Tandur D, Haab L, Podleski L (2018) Intent-based Networks: An Industrial Perspective. In: Proceedings of the 1st International Workshop on Future Industrial Communication Networks (FICN '18). ACM, New York, NY, USA, pp. 35–40
9. Saha BK, Haab L, Podleski L (2022) Intent-based Industrial Network Management Using Natural Language Instructions. IEEE CONECCT 2022:1–6
10. Yuan X, Chen Y (2022) Secure routing protocol based on dynamic reputation and load balancing in wireless mesh networks. J Cloud Comput 11(77). https://doi.org/10.1186/s13677-022-00346-x
11. Abdulrab HQA, Hussin FA, Aziz AA, Awang A, Ismail I, Saat MSM, Shutari H (2022) Optimal Coverage and Connectivity in Industrial Wireless Mesh Networks Based on Harris' Hawk Optimization Algorithm. IEEE Access 10:51048–51061
12. Kostadinovic M, Stjepanovic A, Kuzmic G, Stojcic M, Kostadinovic T (2020) Quality Analysis of Data Transferring Through the Process of Modeling WirelessHART Network. In: 2020 19th International Symposium INFOTEH-JAHORINA (INFOTEH), pp 1–5
13. Hsieh S-Y, Lai C-C (2019) A Novel Scheme for Improving the Reliability in Smart Grid Neighborhood Area Networks. IEEE Access 7:129942–129954
14. Lin XV, Socher R, Xiong C (2020) Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp 4870–4888. Association for Computational Linguistics, Online. https://doi.org/10.18653/v1/2020.findings-emnlp.438. https://aclanthology.org/2020.findings-emnlp.438
15. Wang B, Shin R, Liu X, Polozov O, Richardson M (2020) RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp 7567–7578. Association for Computational Linguistics, Online
16. Kim H, So B-H, Han W-S, Lee H (2021) Natural Language to SQL: Where Are We Today? Proc. VLDB Endow. 13(10):1737–1750
17. Rubin O, Berant J (2021) SmBoP: Semi-autoregressive Bottom-up Semantic Parsing. In: Proceedings of the 5th Workshop on Structured Prediction for NLP (SPNLP 2021). Association for Computational Linguistics, Online, pp 12–21
18. Scholak T, Schucher N, Bahdanau D (2021) PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Online, pp 9895–9901
19. Katsogiannis-Meimarakis G, Koutrika G (2023) A survey on deep learning approaches for text-to-SQL. VLDB J 32:905–936. https://doi.org/10.1007/s00778-022-00776-8
20. Zhang X, Wang Y (2023) DeepMECagent: multi-agent computing resource allocation for UAV-assisted mobile edge computing in distributed IoT system. Appl Intell 53(1):1180–1191. https://doi.org/10.1007/s10489-022-03482-8
21. Saha BK, Haab L, Podleski L (2021) SAFAR: Simulated Annealing-based Flow Allocation Rules for Industrial Networks. IEEE Trans Netw Serv Manag 18(3):3771–3782. https://doi.org/10.1109/TNSM.2020.3035792
22. Khan IA, Pi D, Khan N, Khan ZU, Hussain Y, Nawaz A, Ali F (2021) A privacy-conserving framework based intrusion detection method for detecting and recognizing malicious behaviours in cyber-physical power networks. Appl Intell 51(10):7306–7321. https://doi.org/10.1007/s10489-021-02222-8
23. 2022 Developer Survey. [Accessed: Jan. 24, 2023]. https://survey.stackoverflow.co/2022/#most-popular-technologies-database-prof
24. Michel O, Sonchack J, Keller E, Smith JM (2019) PIQ: Persistent Interactive Queries for Network Security Analytics. In: Proceedings of the ACM International workshop on security in software defined networks & network function virtualization, pp 17–22
25. Wu F, Song HH, Yin J, Gao L, Baldi M, Anand N (2021) NEMA: Automatic Integration of Large Network Management Databases. IEEE Trans Netw Serv Manag 18(3):3783–3797
26. Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, Ma J, Li I, Yao Q, Roman S, Zhang Z, Radev D (2018) Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Brussels, Belgium, pp 3911–3921
27. Herzig J, Nowak PK, Müller T, Piccinno F, Eisenschlos J (2020) TaPas: Weakly Supervised Table Parsing via Pre-training. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, Online, pp 4320-4333
28. Devlin J, Chang M-W, Lee K, Toutanova K (2019) BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short

Papers). Association for Computational Linguistics, Minneapolis, Minnesota, pp 4171–4186
29. Navlakha M (2023) Apple Bans ChatGPT Use by Employees, Report Says. [Accessed: 27 June 2023]. https://mashable.com/article/apple-chatgpt-employee-ban-report
30. McCallum S (2023) ChatGPT Banned in Italy over Privacy Concerns. [Accessed: 27 June 2023]. https://www.bbc.com/news/technology-65139406
31. Finegan-Dollak C, Kummerfeld JK, Zhang L, Ramanathan K, Sadasivam S, Zhang R, Radev D (2018) Improving Text-to-SQL Evaluation Methodology. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Melbourne, Australia, pp. 351-360. https://doi.org/10.18653/v1/P18-1033. https://aclanthology.org/P18-1033
32. Liu Q, Chen B, Guo J, Lou J-G, Zhou B, Zhang D (2021) How Far Are We from Effective Context Modeling? An Exploratory Study on Semantic Parsing in Context. In: Proceedings of the twenty-ninth international joint conference on artificial intelligence, pp 3580–3586
33. Wang R-Z, Ling Z-H, Zhou J-B, Hu Y (2021) A Multiple-Integration Encoder for Multi-Turn Text-to-SQL Semantic Parsing. IEEE/ACM Trans Audio, Speech, Language Process 29:1503–1513. https://doi.org/10.1109/TASLP.2021.3070726

Barun Kumar Saha is a Senior R&D Engineer with Hitachi Energy, Bangalore, India. He received a PhD degree in Computer Science and Engineering from the Indian Institute of Technology Kharagpur, India. His primary areas of work include computer networks, AI, and NLP. His research has been published in several peer-reviewed conferences and journals. He has also co-authored a textbook, *Opportunistic Mobile Networks: Advances and Applications*, published by Springer. Barun was a recipient of the IEEE First Author Awards (IEEE Communications Society, Bangalore Section, 2021). He has also won 3rd Place in the Llama 2 Hackathon with Clarifai (2023). He has delivered keynotes and invited talks at different conferences and universities. Barun is a Member of IEEE. Further details about Barun can be found at https://barunsaha.me/

Paul Gordon has over 35 years of engineering experience with the last 25 spent in Silicon Valley executive engineering roles. Paul is the currently Vice President, R&D for Hitachi Energy Wireless. Prior to that Paul served as Vice President of Engineering for Trilliant Networks, a SmartGrid pioneer, as CEO of Skypilot Networks, as the Vice President of Multimedia Platforms of Pacific Broadband Communications (acquired by Juniper) and as Vice President of Engineering for cable modem leader Com21. He holds several patents in the area of wireless mesh and ATM data transport over cable. He received a B.Eng in Electrical Engineering from the University of Liverpool, UK.

Tore Gillbrand is Vice President of Global Product Management at Hitachi Energy. Tore Gillbrand extends Hitachi Energy's leadership in wireless communication technologies for utilities, oil and gas, and mining operators. A 25-year veteran of the communications industry, Gillbrand joined Hitachi Energy in 2015 and is based at their offices in Santa Clara, California. Prior to that, he served in product management and marketing roles at Cisco Systems and Brocade Communications. Gillbrand holds a Master's degree in Electrical Engineering from Chalmers University of Technology, Gothenburg, Sweden.